

Animate Game Objects



Games need to have identifiable action and elements to enhance gameplay. These actions and elements must be clear for the player from all the angles.



Game development requires a developer to possess creative and visualisation skills, among others. Another important requirement is a game engine that does justice to the game. Depending on the visuals and complexity of the game, the engine needs to keep up and make a good quality game. A huge number of game engines exist for us to choose from. Since we have already learnt about Unity, let's explore it some more.

Recall:

Unity is a cross-platform game engine developed by Unity Technologies. It is used to create three-dimensional (3D) and two-dimensional (2D) games, as well as interactive simulations and other experiences.

12. Animate Game Objects

In the last lesson, we created a GameObject, placed it in a 3D space, and added movement to it with respect to user input.



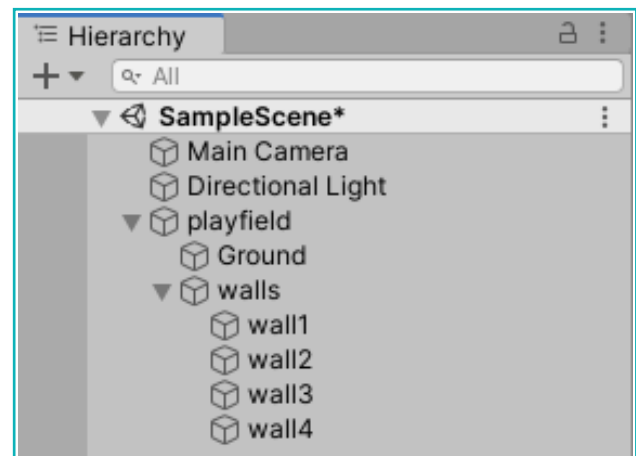
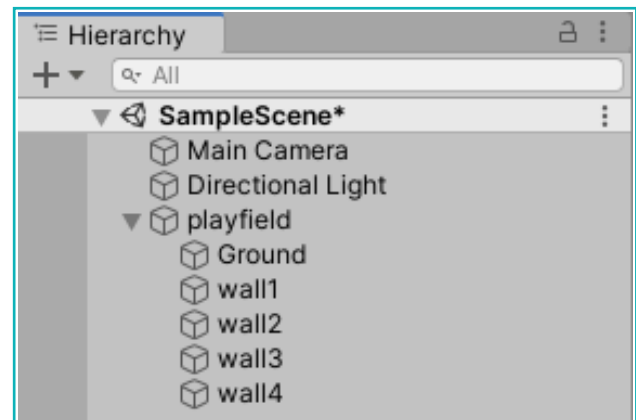
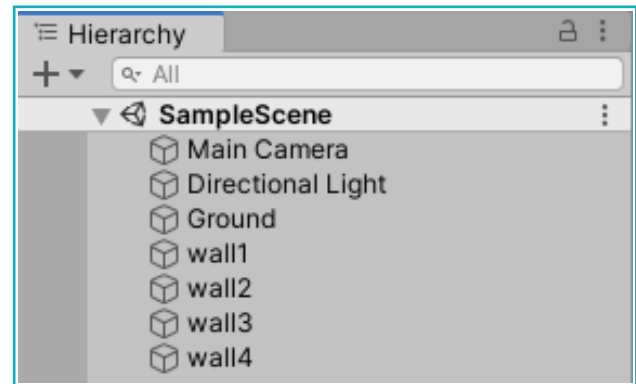
Recall:

GameObjects are fundamental objects in Unity that represent characters, props, and scenery.

As we develop a game, the list of GameObjects gets longer. Managing a long list of GameObjects can be a confusing challenge.

To make our lives simpler, Unity uses the concept of parent-child hierarchies or parenting to group GameObjects. An object can contain other GameObjects which inherit its properties. We can link GameObjects together to move, scale, or transform a collection of them. When we move the top-level object or parent GameObject, all the child GameObjects move with it.

We can create nested parent-child GameObjects. All nested objects are still descendants of the original parent or root GameObject.



12. Animate Game Objects

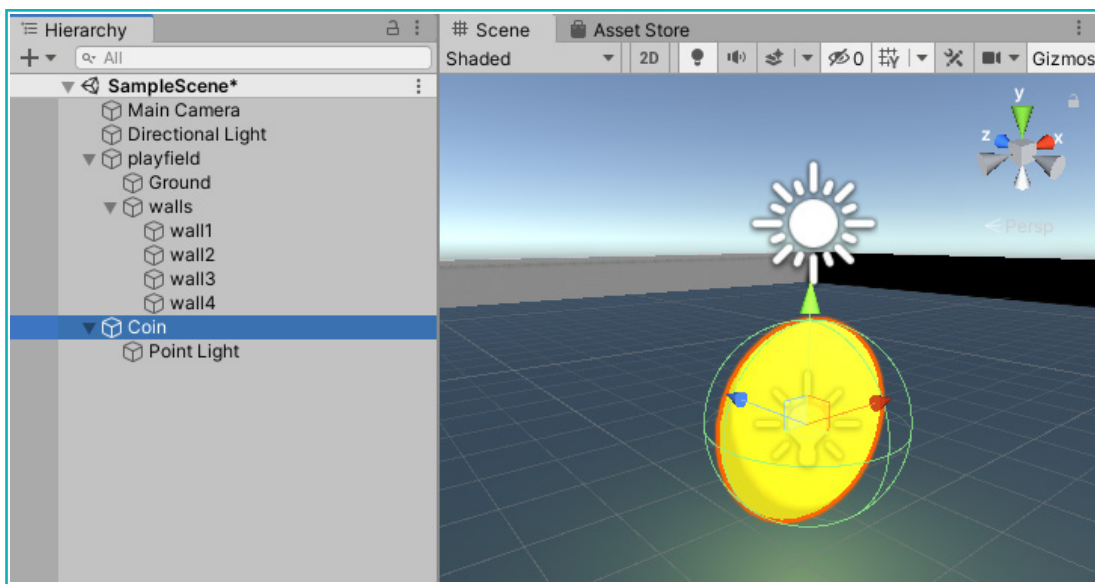
Note:

Child GameObjects inherit the movement and rotation of the parent GameObject.

Animate a GameObject

1. Using a script

Some objects require transformations during a game's runtime. They can be used strategically to grab a player's attention. Here, we have created a coin GameObject using a 3D sphere. The coin has a point light child GameObject which gives it an illuminating effect.

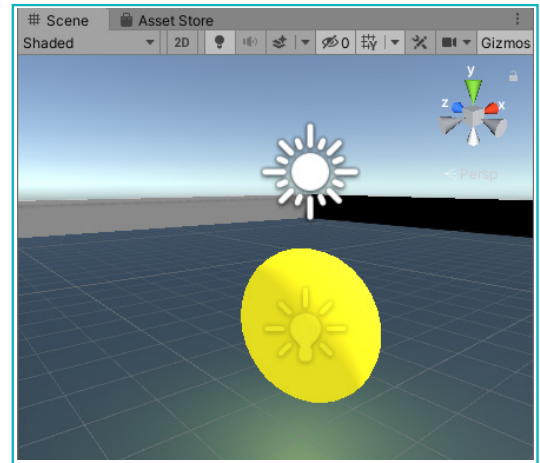


A script is one of the methods at our disposal to keep an object moving throughout the runtime. Here, we have assigned a coin_movement script to the coin GameObject.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class coin_movement : MonoBehaviour
6  {
7      void Update()
8      {
9          transform.Rotate(new Vector3(0, 1.5f, 0));
10     }
11 }
```

12. Animate Game Objects

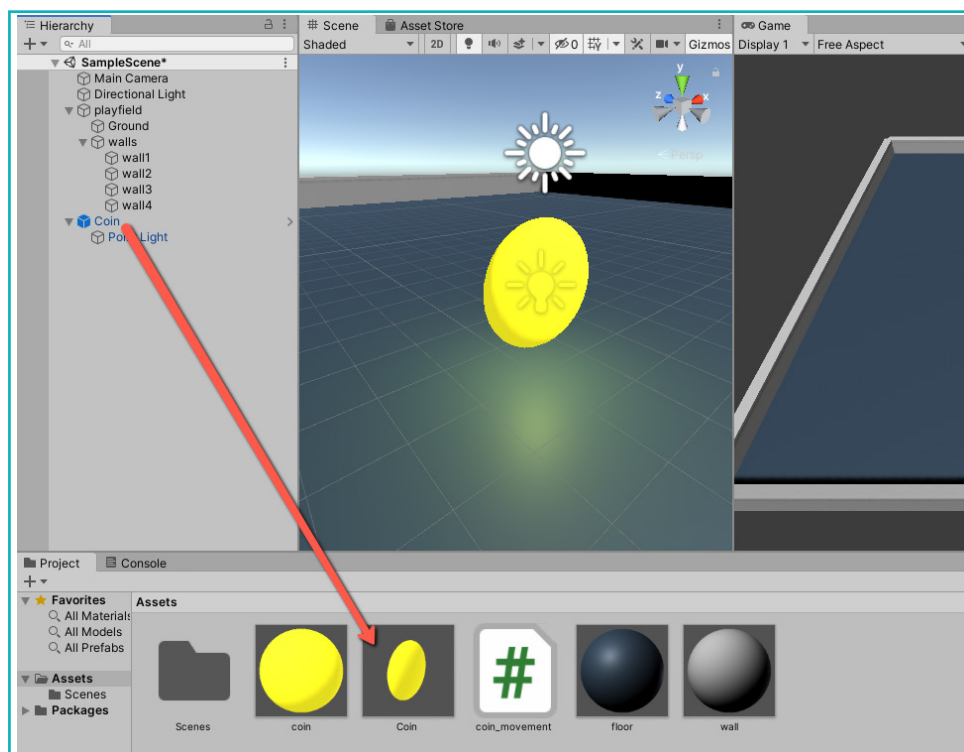
We used the rotate parameter of the transform component to define the rotation in each frame. We have set its rotation along the y-axis. When we start testing the game, the coin GameObject—and its child component—will rotate along the y-axis in each frame.



In games, similar GameObjects are often placed multiple times throughout a level. Most commonly, these are coins, walls, enemies, background elements, etc. It's time-consuming to create the same object each time it is to be placed in the game. When this object needs to be modified, the changes will need to be executed for each of its duplicates separately. The Unity system allows us to create, configure, and store a GameObject with all its components, property values, and child GameObjects as a reusable asset. This kind of reusable asset is called a Prefab.

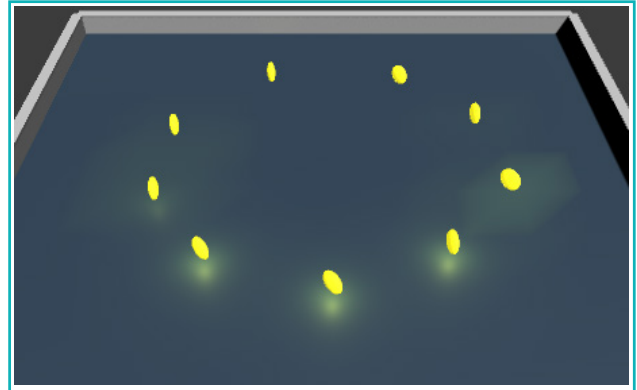
A prefab asset acts as a template to create new prefab instances in the scene. Any edits that we make to a prefab asset are automatically reflected in the instances of that prefab. This enables us to easily make changes in the whole project without manually repeating the same edits in each copy.

To convert an object to a Prefab, select the GameObject in the hierarchy and drag it into the project window.



12. Animate Game Objects

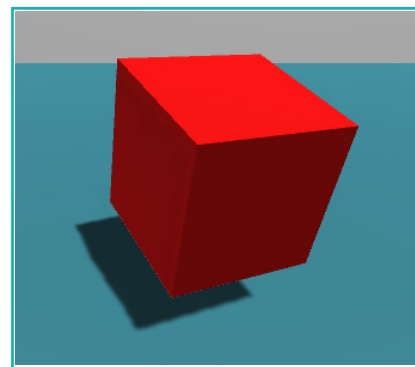
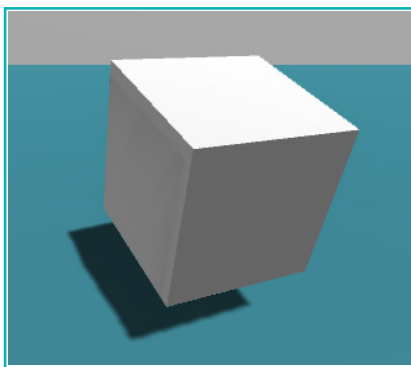
Observe that the GameObject in the hierarchy list appears blue. The GameObject—along with its child GameObjects—which has been converted to a Prefab will appear this way. This Prefab can be used multiple times throughout the game. Any change in the Prefab will reflect in all its instances throughout the game.



Like a transform component, the material of a GameObject can also be changed using a script.

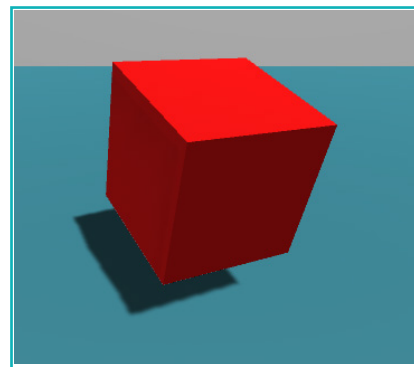
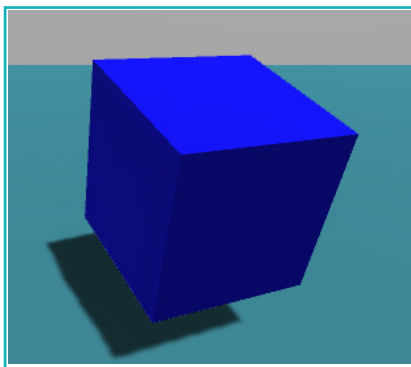
Often, in coding for games, we need to decide the change in appearance based on a condition. For instance, if key R is pressed then turn the box red.

```
color_change.cs x
Assembly-CSharp color_change
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class color_change : MonoBehaviour
6 {
7     void Update()
8     {
9         if (Input.GetKey(KeyCode.R))
10        {
11            GetComponent<Renderer>().material.color = Color.red;
12        }
13    }
14 }
15
```



An extension of the if statement is the if-else statement. The else action will be executed when the first condition is not satisfied.

```
color_change.cs x
Assembly-CSharp color_change
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class color_change : MonoBehaviour
6 {
7     void Update()
8     {
9         if (Input.GetKey(KeyCode.R))
10        {
11            GetComponent<Renderer>().material.color = Color.red;
12        }
13        else
14        {
15            GetComponent<Renderer>().material.color = Color.blue;
16        }
17    }
18 }
```



2. Using Unity's animation tool

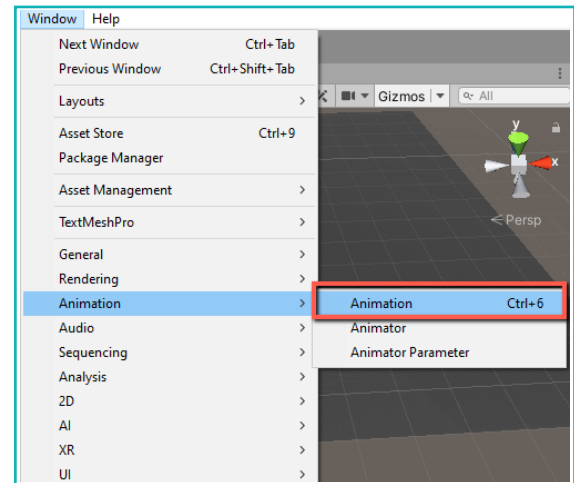
Unity's animation tool is used to create animations in a frame timeline. The animation window provides a standard set of tools necessary for animation. They include keyframes, the playhead, the animation timeline, and curves. These tools allow users to create and adjust the look and feel they desire for their animations.

Animation window: The window in which we create and edit animation clips.

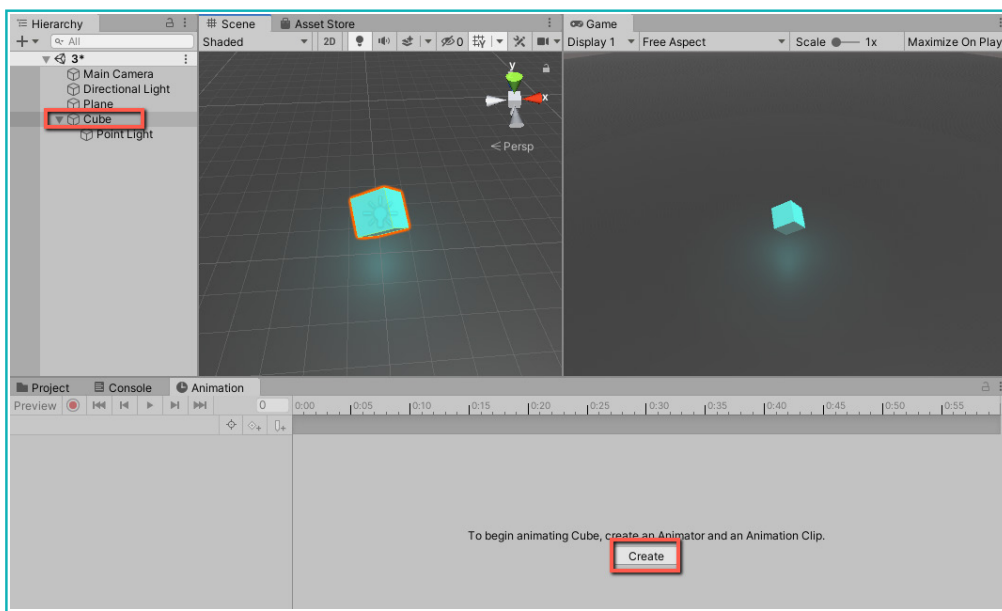
12. Animate Game Objects

Animator window: The grid pane that serves as a visual interface for us to create, modify, and connect animations.

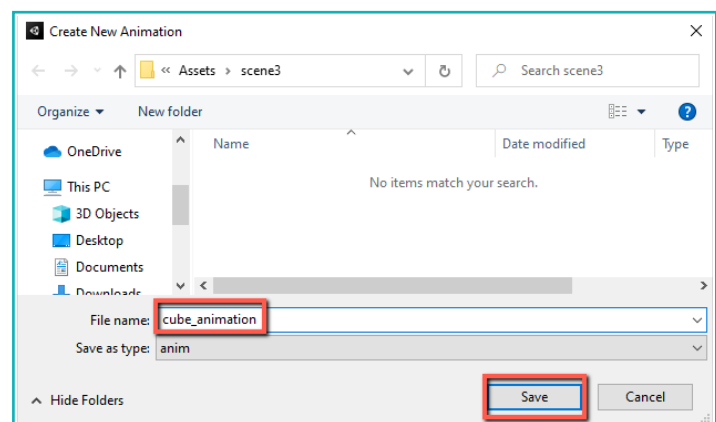
Open the animation window.



Select the object to animate and click on create.

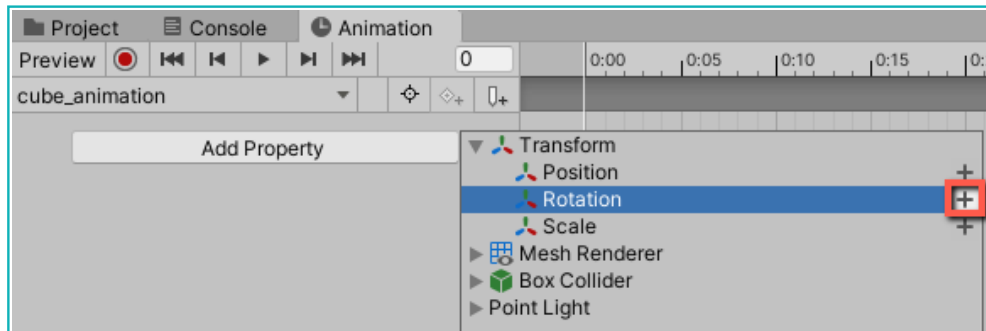


Name the animation and save the file.

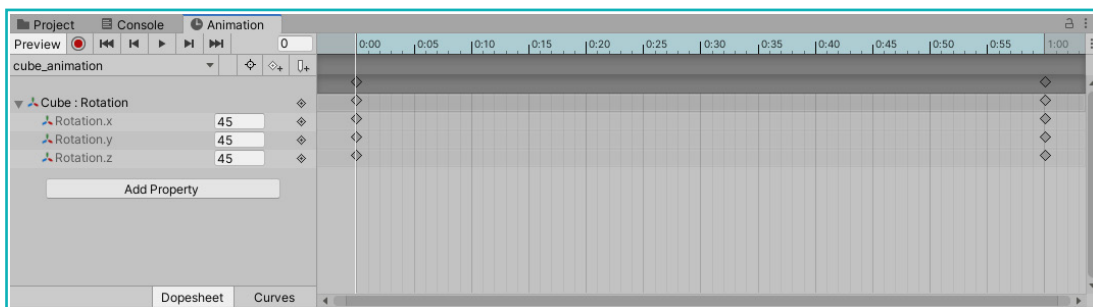


12. Animate Game Objects

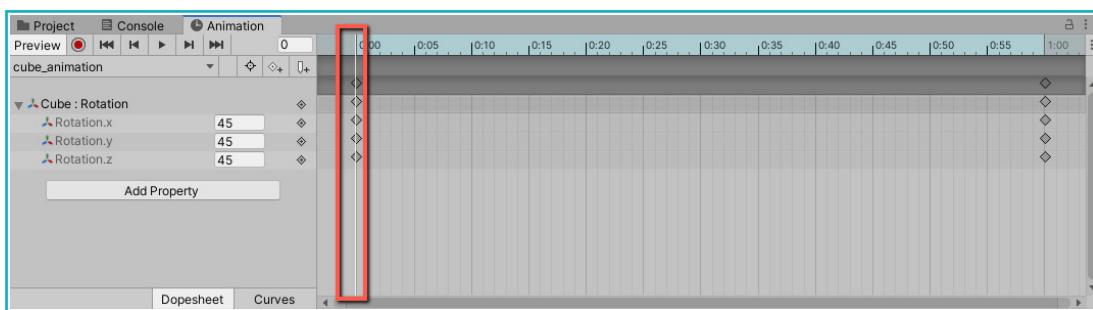
Click on Add Property to add a rotation property.



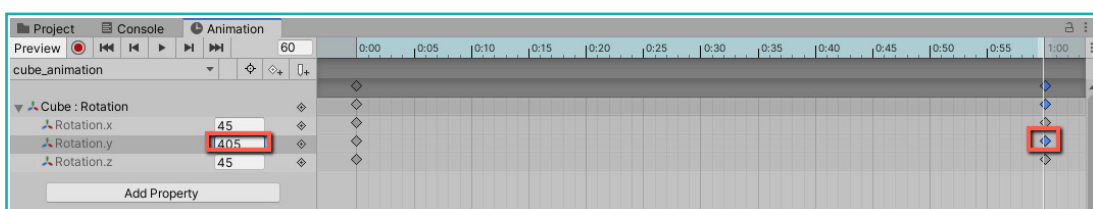
GameObjects can be animated in Unity via the animation window, using traditional keyframe animation techniques. Keyframes are points on an animation clip timeline that contain data about the GameObject, for instance, its transform data. The keyframes indicate some sort of change in that data that produces an animation. When the animation plays, Unity interpolates data from one keyframe to the next.



The white line on the animation timeline is called the Playhead and it indicates where a keyframe will be added. To move it, simply left-click and drag the playhead to the desired frame.

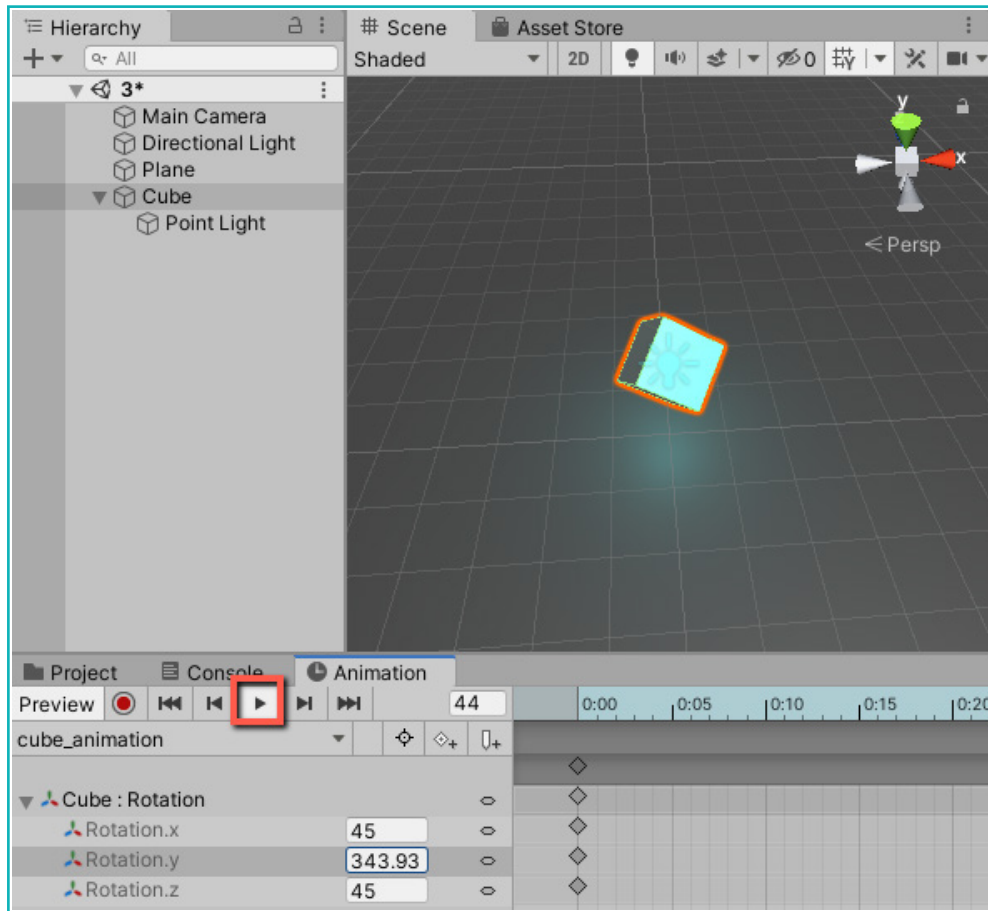


Here, we have set the rotation for the y-axis. In its initial stage, the coin is set at 45 deg. Set the next key at 405 deg and at 1 sec on the timeline.

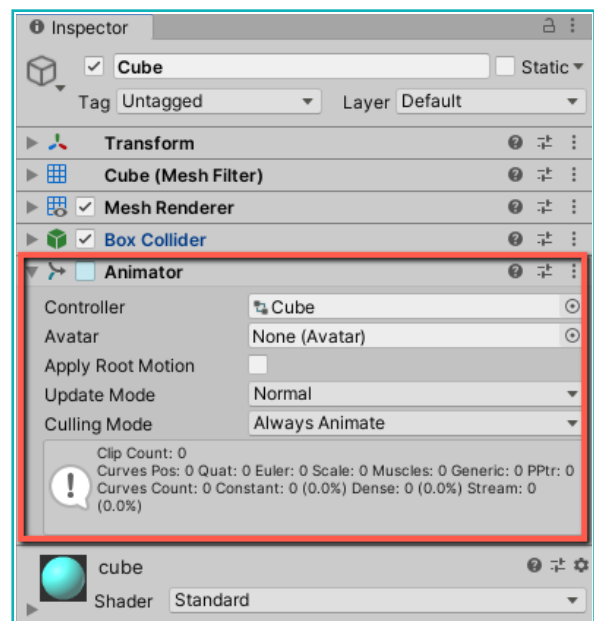


12. Animate Game Objects

Finally, press the play button to view the animation.



As we create an animation property, the animator component is applied in the inspector panel.



Exercise



Answer in one line

1

What is a Prefab?



2

What is a parent-child hierarchy?



3

What is a playhead?



Unity is a popular game engine that enables developers to easily create games and experiences in both 2D and 3D. In this lesson, we learnt about parent-child hierarchies in Unity. We also learned about prefabs, which allow us to reuse GameObjects and their properties without manually creating them again.

12. Animate Game Objects

Animations need to be designed efficiently and elegantly, with respect to the elements in them.



Tech Flash

- **Animation Window** : A window that allows us to create and edit animation clips.
- **Keyframes** : Points on an animation clip timeline that contain data about the GameObject.